**Ancestors_TreeType/TreeType.h**

```
#ifndef TREETYPE_H
#define TREETYPE_H
#include <string>
#include <fstream>
#include "QueType.h"
using namespace std;

enum OrderType {PRE_ORDER, IN_ORDER, POST_ORDER};

template<class ItemType>
struct TreeNode;

template <class ItemType>
class TreeType
{
public:
  TreeType();                    // constructor
 ~TreeType();                   // destructor
  TreeType(const TreeType<ItemType>& originalTree);
  void operator=(const TreeType<ItemType>& originalTree);
  // copy constructor
  void MakeEmpty();
  bool IsEmpty() const;
  bool IsFull() const;
  int GetLength() const;
  ItemType GetItem(ItemType item, bool& found);
  void PutItem(ItemType item);
  void DeleteItem(ItemType item);
  void ResetTree(OrderType order);
  ItemType GetNextItem(OrderType order, bool& finished);
  void Print(ofstream& outFile) const;
  string Ancestors(ItemType item) const;
  string AncestorsNext(TreeNode<ItemType>* node, ItemType item) const;
private:
  TreeNode<ItemType>* root;
  QueType<ItemType> preQue;
  QueType<ItemType> inQue;
  QueType<ItemType> postQue;
};

template <class ItemType>
struct TreeNode
{
  ItemType info;
  TreeNode* left;
  TreeNode* right;
};
```

```
48
49  template <class ItemType>
50  bool TreeType<ItemType>::IsFull() const
51  // Returns true if there is no room for another item
52  //  on the free store; false otherwise.
53  {
54    TreeNode<ItemType>* location;
55    try
56    {
57      location = new TreeNode<ItemType>;
58      delete location;
59      return false;
60    }
61    catch(bad_alloc exception)
62    {
63      return true;
64    }
65  }
66
67  template <class ItemType>
68  bool TreeType<ItemType>::IsEmpty() const
69  // Returns true if the tree is empty; false otherwise.
70  {
71    return root == NULL;
72  }
73
74  template <class ItemType>
75  int CountNodes(TreeNode<ItemType>* tree);
76
77  template <class ItemType>
78  int TreeType<ItemType>::GetLength() const
79  // Calls recursive function CountNodes to count the
80  // nodes in the tree.
81  {
82    return CountNodes(root);
83  }
84
85  template <class ItemType>
86  int CountNodes(TreeNode<ItemType>* tree)
87  // Post: returns the number of nodes in the tree.
88  {
89    if (tree == NULL)
90      return 0;
91    else
92      return CountNodes(tree->left) + CountNodes(tree->right) + 1;
93  }
94
95  template <class ItemType>
96  void Retrieve(TreeNode<ItemType>* tree,
```

```
97      ItemType& item, bool& found);
98
99  template <class ItemType>
100 ItemType TreeType<ItemType>::GetItem(ItemType item, bool& found)
101 // Calls recursive function Retrieve to search the tree for item.
102 {
103   Retrieve(root, item, found);
104   return item;
105 }
106
107 template <class ItemType>
108 void Retrieve(TreeNode<ItemType>* tree,
109      ItemType& item, bool& found)
110 // Recursively searches tree for item.
111 // Post: If there is an element someItem whose key matches item's,
112 //       found is true and item is set to a copy of someItem;
113 //       otherwise found is false and item is unchanged.
114 {
115   if (tree == NULL)
116     found = false;                     // item is not found.
117   else if (item < tree->info)
118     Retrieve(tree->left, item, found); // Search left subtree.
119   else if (item > tree->info)
120     Retrieve(tree->right, item, found);// Search right subtree.
121   else
122   {
123     item = tree->info;                 // item is found.
124     found = true;
125    }
126 }
127
128 template <class ItemType>
129 void Insert(TreeNode<ItemType>*& tree, ItemType item);
130
131 template <class ItemType>
132 void TreeType<ItemType>::PutItem(ItemType item)
133 // Calls recursive function Insert to insert item into tree.
134 {
135   Insert(root, item);
136 }
137
138 template <class ItemType>
139 void Insert(TreeNode<ItemType>*& tree, ItemType item)
140 // Inserts item into tree.
141 // Post:  item is in tree; search property is maintained.
142 {
143   if (tree == NULL)
144   {// Insertion place found.
145     tree = new TreeNode<ItemType>;
```

```
146     tree->right = NULL;
147     tree->left = NULL;
148     tree->info = item;
149   }
150   else if (item < tree->info)
151     Insert(tree->left, item);    // Insert in left subtree.
152   else
153     Insert(tree->right, item);   // Insert in right subtree.
154 }
155
156 template <class ItemType>
157 void DeleteNode(TreeNode<ItemType>*& tree);
158
159 template <class ItemType>
160 void Delete(TreeNode<ItemType>*& tree, ItemType item);
161
162 template <class ItemType>
163 void TreeType<ItemType>::DeleteItem(ItemType item)
164 // Calls recursive function Delete to delete item from tree.
165 {
166   Delete(root, item);
167 }
168
169 template <class ItemType>
170 void Delete(TreeNode<ItemType>*& tree, ItemType item)
171 // Deletes item from tree.
172 // Post:  item is not in tree.
173 {
174   if (item < tree->info)
175     Delete(tree->left, item);   // Look in left subtree.
176   else if (item > tree->info)
177     Delete(tree->right, item);  // Look in right subtree.
178   else
179     DeleteNode(tree);           // Node found; call DeleteNode.
180 }
181
182 template <class ItemType>
183 void GetPredecessor(TreeNode<ItemType>* tree, ItemType& data);
184
185 template <class ItemType>
186 void DeleteNode(TreeNode<ItemType>*& tree)
187 // Deletes the node pointed to by tree.
188 // Post: The user's data in the node pointed to by tree is no
189 //       longer in the tree.  If tree is a leaf node or has only
190 //       non-NULL child pointer the node pointed to by tree is
191 //       deleted; otherwise, the user's data is replaced by its
192 //       logical predecessor and the predecessor's node is deleted.
193 {
194   ItemType data;
```

```
195   TreeNode<ItemType>* tempPtr;
196
197   tempPtr = tree;
198   if (tree->left == NULL)
199   {
200     tree = tree->right;
201     delete tempPtr;
202   }
203   else if (tree->right == NULL)
204   {
205     tree = tree->left;
206     delete tempPtr;
207   }
208   else
209   {
210     GetPredecessor(tree->left, data);
211     tree->info = data;
212     Delete(tree->left, data);  // Delete predecessor node.
213   }
214 }
215
216 template <class ItemType>
217 void GetPredecessor(TreeNode<ItemType>* tree, ItemType& data)
218 // Sets data to the info member of the right-most node in tree.
219 {
220   while (tree->right != NULL)
221     tree = tree->right;
222   data = tree->info;
223 }
224
225 template <class ItemType>
226 void PrintTree(TreeNode<ItemType>* tree, ofstream& outFile)
227 // Prints info member of items in tree in sorted order on outFile.
228 {
229   if (tree != NULL)
230   {
231     PrintTree(tree->left, outFile);   // Print left subtree.
232     outFile << tree->info;
233     PrintTree(tree->right, outFile);  // Print right subtree.
234   }
235 }
236
237 template <class ItemType>
238 void TreeType<ItemType>::Print(ofstream& outFile) const
239 // Calls recursive function Print to print items in the tree.
240 {
241   PrintTree(root, outFile);
242 }
243
```

```
244 template <class ItemType>
245 TreeType<ItemType>::TreeType()
246 {
247   root = NULL;
248 }
249
250 template <class ItemType>
251 void Destroy(TreeNode<ItemType>*& tree);
252
253 template <class ItemType>
254 TreeType<ItemType>::~TreeType()
255 // Calls recursive function Destroy to destroy the tree.
256 {
257   Destroy(root);
258 }
259
260 template <class ItemType>
261 void Destroy(TreeNode<ItemType>*& tree)
262 // Post: tree is empty; nodes have been deallocated.
263 {
264   if (tree != NULL)
265   {
266     Destroy(tree->left);
267     Destroy(tree->right);
268     delete tree;
269   }
270 }
271
272 template <class ItemType>
273 void TreeType<ItemType>::MakeEmpty()
274 {
275   Destroy(root);
276   root = NULL;
277 }
278
279 template <class ItemType>
280 void CopyTree(TreeNode<ItemType>*& copy, const TreeNode<ItemType>* originalTree);
281
282 template <class ItemType>
283 TreeType<ItemType>::TreeType(const TreeType<ItemType>& originalTree)
284 // Calls recursive function CopyTree to copy originalTree
285 //  into root.
286 {
287   CopyTree(root, originalTree.root);
288 }
289
290 template <class ItemType>
291 void TreeType<ItemType>::operator=
292      (const TreeType& originalTree)
```

```
293 // Calls recursive function CopyTree to copy originalTree
294 // into root.
295 {
296   {
297   if (&originalTree == this)
298     return;             // Ignore assigning self to self
299   Destroy(root);      // Deallocate existing tree nodes
300   CopyTree(root, originalTree.root);
301   }
302 
303 }
304 
305 template <class ItemType>
306 void CopyTree(TreeNode<ItemType>*& copy,
307      const TreeNode<ItemType>* originalTree)
308 // Post: copy is the root of a tree that is a duplicate
309 //       of originalTree.
310 {
311   if (originalTree == NULL)
312     copy = NULL;
313   else
314   {
315     copy = new TreeNode<ItemType>;
316     copy->info = originalTree->info;
317     CopyTree(copy->left, originalTree->left);
318     CopyTree(copy->right, originalTree->right);
319   }
320 }
321 
322 // Function prototypes for auxiliary functions.
323 template <class ItemType>
324 void PreOrder(TreeNode<ItemType>*, QueType<ItemType>&);
325 // Enqueues tree items in preorder.
326 
327 template <class ItemType>
328 void InOrder(TreeNode<ItemType>*, QueType<ItemType>&);
329 // Enqueues tree items in inorder.
330 
331 template <class ItemType>
332 void PostOrder(TreeNode<ItemType>*, QueType<ItemType>&);
333 // Enqueues tree items in postorder.
334 
335 
336 template <class ItemType>
337 void TreeType<ItemType>::ResetTree(OrderType order)
338 // Calls function to create a queue of the tree elements in
339 // the desired order.
340 {
341   switch (order)
```

```
342   {
343     case PRE_ORDER : PreOrder(root, preQue);
344                      break;
345     case IN_ORDER  : InOrder(root, inQue);
346                      break;
347     case POST_ORDER: PostOrder(root, postQue);
348                      break;
349   }
350 }
351
352 template <class ItemType>
353 void PreOrder(TreeNode<ItemType>* tree,
354      QueType<ItemType>& preQue)
355 // Post: preQue contains the tree items in preorder.
356 {
357   if (tree != NULL)
358   {
359     preQue.Enqueue(tree->info);
360     PreOrder(tree->left, preQue);
361     PreOrder(tree->right, preQue);
362   }
363 }
364
365 template <class ItemType>
366 void InOrder(TreeNode<ItemType>* tree,
367      QueType<ItemType>& inQue)
368 // Post: inQue contains the tree items in inorder.
369 {
370   if (tree != NULL)
371   {
372     InOrder(tree->left, inQue);
373     inQue.Enqueue(tree->info);
374     InOrder(tree->right, inQue);
375   }
376 }
377
378 template <class ItemType>
379 void PostOrder(TreeNode<ItemType>* tree,
380      QueType<ItemType>& postQue)
381 // Post: postQue contains the tree items in postorder.
382 {
383   if (tree != NULL)
384   {
385     PostOrder(tree->left, postQue);
386     PostOrder(tree->right, postQue);
387     postQue.Enqueue(tree->info);
388   }
389 }
390
```

```
391 template <class ItemType>
392 ItemType TreeType<ItemType>::GetNextItem(OrderType order, bool& finished)
393 // Returns the next item in the desired order.
394 // Post: For the desired order, item is the next item in the queue.
395 //       If item is the last one in the queue, finished is true;
396 //       otherwise finished is false.
397 {
398   finished = false;
399   ItemType item;
400   switch (order)
401   {
402     case PRE_ORDER  : preQue.Dequeue(item);
403                       if (preQue.IsEmpty())
404                         finished = true;
405                       break;
406     case IN_ORDER   : inQue.Dequeue(item);
407                       if (inQue.IsEmpty())
408                         finished = true;
409                       break;
410     case  POST_ORDER: postQue.Dequeue(item);
411                       if (postQue.IsEmpty())
412                         finished = true;
413                       break;
414   }
415   return item;
416 }
417
418 // Ivan Temesvari
419 // 3/11/2019
420 // This recursive function will return the ancestors of item in reverse order.
421 template <class ItemType>
422 string TreeType<ItemType>::Ancestors(ItemType item) const{
423     //complete this
424 }
425
426 template <class ItemType>
427 string TreeType<ItemType>::AncestorsNext(TreeNode<ItemType>* node, ItemType item)
    const{
428     //Base case: the item node is found
429
430     //Recursive case: go down to the next ancestor/parent: top-left or top-right
    parent?
431
432 }
433 #endif
434
```